

PREPILA DYSON

Onde o sol nunca se põe

INTEGRANTES

-  Victor Ulrich Costa Alves da Silva - 568634
-  Arthur da Silva Santana - 571075
-  Matheus Pereira da Silva Franco - 569315
-  Matheus Luca Fouad Barragao - 572228
-  Yasmin Cappi Santos - 571926

CURSO: Análise e Desenvolvimento de Sistemas

TURMA: 1TDSPW

PROFESSOR: Fabio Rogerio Lins Pereira de Souza

DISCIPLINA: Computacional Thinking Using Python

DATA: 2026

Sumario

Sumario.....	1
Documentacao do Projeto Python	3
Descricao da solucao.....	3
Conceitos técnicos aplicados.....	3
Estrutura do projeto	5
Menu do sistema	6
Como executar	7
Regras de negócio implementadas.....	9
Prints	9

Documentação do Projeto Python

Descrição da solução

O Prepila Dyson é uma plataforma de inteligência operacional voltada para o gerenciamento de futuras infraestruturas de Energia Solar Espacial (SBSP — Space-Based Solar Power). Conforme satélites equipados com painéis solares passam a transmitir energia para estações receptoras terrestres, surge a necessidade de uma camada centralizada capaz de monitorar a operação, integrar dados de múltiplas fontes, detectar anomalias, emitir alertas e apoiar a tomada de decisão.

Este projeto entrega o protótipo funcional em Python dessa plataforma: um sistema executado via terminal, com menu interativo, que simula o fluxo operacional completo — cadastro de satélites e estações, registro de monitoramentos de energia, detecção automática de anomalias, geração de alertas priorizados, relatórios analíticos com estatísticas operacionais, recomendações geradas por IA simulada e busca/filtros sobre os dados. O Python é usado para automatizar a análise dos dados orbitais, calcular eficiência energética e classificar a severidade das ocorrências, transformando dados brutos em informação estratégica.

A solução conecta-se diretamente ao tema da Global Solution (economia espacial / nova fronteira tecnológica) e contribui para os ODS 7, 9 e 13 (energia limpa, inovação e ação climática).

Conceitos técnicos aplicados

Conceito	Onde aparece no código
if-elif-else	severidade_para_prioridade(), validações de status em registrar_monitoramento()
match-case	classificar_eficiencia(), buscar_satelites_por_filtro(), seleção de opções no menu principal (main())
while	Loop principal do sistema em main(), além das funções de validação ler_texto_obrigatorio(),

	ler_numero_float(), ler_opcao_de_tupla() e ler_inteiro()
for	Iterações em detectar_anomalias_e gerar_alertas(), gerar_relatorio_analitico(), gerar_recomendacao_ia(), listar_recursos() e visualizar_alertas()
Listas (list)	Estrutura principal de armazenamento dos dados: satélites, estações, monitoramentos, anomalias, alertas e recomendações
Tuplas (tuple)	Constantes imutáveis utilizadas para validação: STATUS_SATELITE, STATUS_ESTACAO, SEVERIDADES e PRIORIDADES_ALERTA
Strings (str)	Manipulação de IDs, nomes, organizações, localizações, mensagens do sistema e métodos como lower(), upper(), strip() e operador in
Funções com parâmetros e retorno	gerar_proximo_id(), classificar_eficiencia(), buscar_por_id(), gerar_relatorio_analitico(), gerar_recomendacao_ia(), entre outras

Procedimentos (funções sem retorno relevante)

exibir_menu(), exibir_titulo(),
exibir_subtitulo(), listar_recursos() e
visualizar_alertas()

def / import

Todas as funcionalidades foram implementadas com def; o arquivo main.py importa constantes e dados do módulo dados_iniciais.py

Validação de entrada do usuário

Implementada por meio das funções ler_texto_obrigatorio(), ler_numero_float(), ler_opcao_de_tupla() e ler_inteiro()

Dicionários (dict)

Utilizados para representar entidades do sistema, como satélites, estações, monitoramentos, anomalias, alertas e recomendações

Manipulação de dados

Cadastro, consulta, busca, filtragem e geração de relatórios operacionais sobre os dados armazenados

Modularização

Separação entre main.py (lógica do sistema) e dados_iniciais.py (dados simulados e constantes)

Menu interativo

Implementado na função main(), permitindo navegação pelas funcionalidades através de opções numéricas

Estrutura do projeto

python-gs/

|— main.py # Programa principal com o menu e todas as funcionalidades
|— dados_iniciais.py # Dados mock (satélites, estações, monitoramentos,
anomalias, alertas)

Menu do sistema

O menu principal possui 10 opções funcionais + saída (o requisito mínimo é 5 itens, dos quais 4 devem ser protótipos funcionais relacionados à GS):

Opção	Funcionalidade	Descrição rápida
1	Sobre a solução	Exibe uma descrição resumida do Prepila Dyson e seus objetivos.
2	Cadastrar satélite	Permite cadastrar um novo satélite na plataforma informando nome, organização, status e capacidade de geração.
3	Cadastrar estação receptora	Permite cadastrar uma nova estação terrestre responsável pelo recebimento da energia transmitida pelos satélites.
4	Registrar monitoramento (energia)	Registra uma leitura operacional contendo energia captada, energia transmitida, data/hora e recursos envolvidos.

5	Detectar anomalias e gerar alertas	Analisa os monitoramentos cadastrados, calcula a eficiência energética e gera anomalias e alertas automaticamente.
6	Listar recursos cadastrados	Exibe em formato tabular todos os satélites e estações receptoras cadastrados na plataforma.
7	Relatório analítico	Apresenta indicadores operacionais, como energia total captada, energia transmitida, eficiência média e quantidade de anomalias por severidade.
8	Recomendação IA	Gera recomendações operacionais baseadas nos dados monitorados e nas anomalias detectadas.
9	Buscar/filtrar satélites	Permite localizar satélites por nome, organização responsável ou status operacional.
10	Visualizar alertas	Exibe todos os alertas registrados, ordenados por nível de prioridade (crítica, alta, média ou baixa).
0	Sair	Encerra a execução do sistema de forma segura.

Como executar

Pré-requisitos

Python 3.10 ou superior (o programa usa match-case). Verifique com:

```
python --version
```

Passo a passo

Abra um terminal na pasta do projeto (python-gs/).

Execute:

```
python main.py
```

No Windows, caso o comando acima abra a Microsoft Store, use:

```
py main.py
```

Use o menu interativo digitando o número da opção desejada.

Sequência sugerida para a apresentação / correção

Opção 1 — apresentar a descrição da solução.

Opção 6 — mostrar os dados pré-carregados (satélites e estações).

Opção 5 — rodar a detecção de anomalias sobre os dados iniciais (gera alertas críticos e altos automaticamente).

Opção 7 — exibir o relatório analítico.

Opção 8 — exibir as recomendações da IA simulada.

Opção 9 — fazer um filtro (por exemplo, status ativo).

Opção 2/3/4 — cadastrar um satélite, uma estação e um monitoramento, depois voltar à 5 e à 7 para mostrar que a plataforma reagiu aos novos dados.

Opção 10 — visualizar todos os alertas ordenados por prioridade.

Opção 0 — sair.

Regras de negócio implementadas

Apenas satélites com status ativo podem participar de operações.

Apenas estações com status ativa podem participar de operações.

Energia captada e transmitida não podem ser negativas (validado na entrada).

A severidade da anomalia é definida pela faixa de eficiência da transmissão:

$\geq 90\%$ → sem anomalia

$\geq 75\%$ → severidade baixo

$\geq 50\%$ → severidade medio

$\geq 25\%$ → severidade alto

$< 25\%$ → severidade critico

Toda anomalia detectada gera automaticamente um alerta com prioridade proporcional à severidade.

Prints

```
# =====  
# UTILITARIOS DE INTERFACE (USABILIDADE)  
# =====  
  
def exibir_titulo(texto):  
    """Imprime um titulo formatado em destaque.  
  
    Parametros:  
    | texto (str): texto que sera exibido no titulo.  
    Retorno:  
    | None (apenas imprime na tela).  
    """  
    linha = "=" * 60  
    print("\n" + linha)  
    print(texto.center(60))  
    print(linha)  
  
def exibir_subtitulo(texto):  
    """Imprime um subtitulo formatado, com largura padronizada."""  
    print("\n" + "-" * 60)  
    print(f" {texto}")  
    print("-" * 60)  
  
def pausa():  
    """Pausa a execucao ate o usuario pressionar ENTER."""  
    input("\n>> Pressione ENTER para voltar ao menu...")
```

```

def cadastrar_estacao(estacoes):
    """Cadastra uma nova estacao receptora.

    Parametros:
    |   estacoes (list): lista de estacoes ja cadastradas.
    Retorno:
    |   dict: estacao recém cadastrada.
    """
    exibir_titulo("CADASTRO DE ESTACAO RECEPTORA")
    nome = ler_texto_obrigatorio("Nome da estacao")
    organizacao = ler_texto_obrigatorio("Organizacao responsavel")
    localizacao = ler_texto_obrigatorio("Localizacao (cidade-pais)")
    status = ler_opcao_de_tupla("Status operacional", STATUS_ESTACAO)

    nova = {
        "id": gerar_proximo_id(estacoes, "EST"),
        "nome": nome,
        "organizacao": organizacao,
        "status": status,
        "localizacao": localizacao,
    }
    estacoes.append(nova)
    print(f"\n[OK] Estacao cadastrada com sucesso! ID gerado: {nova['id']}")
    return nova

```

```

# =====
# CADASTROS (SATELITES E ESTACOES)
# =====

def cadastrar_satelite(satelites):
    """Cadastra um novo satelite na plataforma.

    Parametros:
    |   satelites (list): lista de satelites ja cadastrados.
    Retorno:
    |   dict: satelite recém cadastrado.
    """

    exibir_titulo("CADASTRO DE SATELITE")
    nome = ler_texto_obrigatorio("Nome do satelite")
    organizacao = ler_texto_obrigatorio("Organizacao responsavel")
    status = ler_opcao_de_tupla("Status operacional", STATUS_SATELITE)
    capacidade = ler_numero_float("Capacidade de geracao em kW", 0.1, 10_000.0)

    novo = {
        "id": gerar_proximo_id(satelites, "SAT"),
        "nome": nome,
        "organizacao": organizacao,
        "status": status,
        "capacidade_kw": capacidade,
    }
    satelites.append(novo)
    print(f"\n[OK] Satelite cadastrado com sucesso! ID gerado: {novo['id']}")
    return novo

```

```

# =====
# LISTAGEM DE RECURSOS
# =====

def listar_recursos(satelites, estacoes):
    """Mostra todos os satelites e estacoes cadastrados em formato tabular."""
    exibir_titulo("RECURSOS CADASTRADOS")

    exibir_subtitulo(f"SATELITES ({len(satelites)})")
    if not satelites:
        print("Nenhum satellite cadastrado.")
    else:
        print(f"{'ID':<8} {'NOME':<15} {'ORG':<8} {'STATUS':<15} {'CAP.(kW)':>10}")
        for s in satelites:
            print(
                f"{s['id']:<8} {s['nome']:<15} {s['organizacao']:<8} "
                f"{s['status']:<15} {s['capacidade_kw']:>10.1f}"
            )

    exibir_subtitulo(f"ESTACOES RECEPTORAS ({len(estacoes)})")
    if not estacoes:
        print("Nenhuma estacao cadastrada.")
    else:
        print(f"{'ID':<8} {'NOME':<20} {'ORG':<8} {'STATUS':<15} {'LOCAL':<15}")
        for e in estacoes:
            print(
                f"{e['id']:<8} {e['nome']:<20} {e['organizacao']:<8} "
                f"{e['status']:<15} {e['localizacao']:<15}"
            )

```

```

# =====
# DETECCAO DE ANOMALIAS E ALERTAS
# =====

def classificar_eficiencia(eficiencia):
    """Classifica a eficiencia de transmissao em uma faixa de severidade.

    Usa match-case (Python 3.10+) para selecionar a categoria com base
    em faixas de eficiencia (energia transmitida / energia captada).

    Parametros:
    | eficiencia (float): valor entre 0.0 e 1.0.
    Retorno:
    | tuple[str, str]: (severidade, descricao) - severidade pertence
    | a SEVERIDADES (ou 'ok' quando sem anomalia).
    """
    faixa = int(eficiencia * 100)
    match faixa:
        case f if f >= 90:
            return ("ok", "Operacao dentro do esperado")
        case f if f >= 75:
            return ("baixo", "Pequena perda de eficiencia detectada")
        case f if f >= 50:
            return ("medio", "Perda moderada de eficiencia (entre 25% e 50%)")
        case f if f >= 25:
            return ("alto", "Perda de eficiencia de transmissao acima de 25%")
        case _:
            return ("critico", "Perda de eficiencia de transmissao acima de 50%")

def severidade_para_prioridade(severidade):
    """Converte a severidade da anomalia em prioridade de alerta.

    Usa if/elif/else para mapear severidade -> prioridade do alerta.
    Parametros:
    | severidade (str): valor pertencente a SEVERIDADES.
    Retorno:
    | str: prioridade do alerta.
    """
    if severidade == "critico":
        return "critica"
    elif severidade == "alto":
        return "alta"
    elif severidade == "medio":
        return "media"
    else:
        return "baixa"

```

```
# =====
# VISUALIZACAO DE ALERTAS
# =====

def visualizar_alertas(alertas, anomalias):
    """Lista os alertas operacionais ordenados por prioridade."""
    exibir_titulo("ALERTAS OPERACIONAIS")

    if not alertas:
        print("Nenhum alerta gerado ate o momento.")
        return

    ordem = {"critica": 0, "alta": 1, "media": 2, "baixa": 3}
    alertas_ordenados = sorted(alertas, key=lambda a: ordem.get(a["prioridade"], 99))

    print(f"{'ID':<8} {'PRIORIDADE':<10} {'LIDO':<5} {'DATA/HORA':<18} {'DESCRICAO'}")
    for alerta in alertas_ordenados:
        anomalia = buscar_por_id(anomalias, alerta["anomalia_id"])
        descricao = anomalia["descricao"] if anomalia else "(anomalia nao encontrada)"
        lido = "Sim" if alerta["lido"] else "Nao"
        print(
            f"{'ID':<8} {'PRIORIDADE':<10} {'LIDO':<5} "
            f"{'DATA/HORA':<18} {'DESCRICAO'}"
        )
    )
```

```
=====
                        PREPILA DYSON - MENU PRINCIPAL
=====

1 - Sobre a solucao
2 - Cadastrar satelite
3 - Cadastrar estacao receptora
4 - Registrar monitoramento (energia)
5 - Detectar anomalias e gerar alertas
6 - Listar recursos cadastrados
7 - Relatorio analitico
8 - Recomendacao IA
9 - Buscar/filtrar satelites
10 - Visualizar alertas
0 - Sair

Digite a opcao desejada: █
```

```
=====
                        ALERTAS OPERACIONAIS
=====
ID      PRIORIDADE LIDO  DATA/HORA      DESCRICAO
ALE-002 critica     Nao  2026-05-27 11:05  Perda de eficiencia de transmissao acima de 50%
ALE-001 alta       Nao  2026-05-27 10:05  Perda de eficiencia de transmissao acima de 25%

>> Pressione ENTER para voltar ao menu...
```

```
=====
RECOMENDACAO IA - PREPILA DYSON
=====
```

1. [CRITICO] Satelite Aether-EU (SAT-004) com eficiencia media de 37.0%. Recomenda-se inspecao tecnica imediata.
2. [ACAO] 1 anomalia(s) critica(s) ativa(s) - redistribuir carga para satelites com maior eficiencia.
3. [OPERACIONAL] Satelites fora de operacao: Helios-2. Avaliar retorno ao status ativo apos manutencao.

```
>> Pressione ENTER para voltar ao menu...
```